

Orbit: Model Predictive Control for Adaptive Request Routing in Disaggregated LLM Serving

Tejeshwar Iyengar¹ and Ravi Gupta¹

AMD Corporation
{tejeshwar.iyengar, ravi.gupta}@amd.com

Abstract. Large Language Model (LLM) inference at scale increasingly adopts prefill-decode disaggregation, where specialized server pools handle compute-intensive prompt processing and memory-bound token generation separately. This architecture introduces request routing challenges that traditional load balancers handle poorly: heterogeneous service times, variable workloads, and capacity asymmetries across servers. We present **Orbit**, a Model Predictive Control (MPC) framework that augments standard routing policies with horizon-based prediction and adaptive weight adjustment. Rather than replacing existing algorithms, Orbit wraps them with a predictive control layer that anticipates future congestion. We validate Orbit on a production-class cluster of AMD Instinct MI300X GPUs running vLLM with NixlConnector-based RDMA KV-cache transfer. Across 40 experiments spanning five evaluation phases with the Qwen2.5-14B model, we find that Orbit’s value depends strongly on deployment context: in homogeneous configurations, existing policies like Power-of-Two-Choices (PO2) suffice, and MPC adds 2–5% overhead. However, when server capacities are heterogeneous, MPC-augmented routing achieves **45.8% lower tail inter-token latency** at 3-prefill/3-decode scale and **49.8% lower tail latency** for prefill-heavy workloads. We further show that PO2 is a surprisingly strong baseline, reducing tail latency by 66% over Round Robin even without MPC. These results provide concrete deployment guidance: enable MPC when heterogeneity or workload variability is present, and rely on PO2 otherwise.

Keywords: Request Scheduling · Load Balancing · Model Predictive Control · LLM Serving · Disaggregated Inference · Heterogeneous Clusters

1 Introduction

The deployment of Large Language Models (LLMs) in production systems has driven rapid evolution in inference infrastructure [1, 2]. As models scale to hundreds of billions of parameters and serve millions of daily requests, efficient request scheduling becomes critical for meeting latency objectives while maximizing hardware utilization. A key architectural pattern emerging in large-scale deployments is *prefill-decode disaggregation* [3, 4], where the two phases

of autoregressive generation—prompt processing (prefill) and token generation (decode)—are handled by separate, specialized server pools.

This separation is motivated by the distinct computational characteristics of each phase. Prefill is compute-bound: processing input tokens in parallel requires high arithmetic throughput. Decode is memory-bound: generating tokens autoregressively requires repeated KV-cache access with limited compute per step. By assigning these phases to appropriately configured hardware, disaggregation improves overall efficiency. However, it introduces a routing challenge: requests must be directed first to a prefill server, then transferred (along with their KV-cache state) to a decode server, with each routing decision affecting end-to-end latency.

Traditional load balancing algorithms—Round Robin (RR), Least Connections, Power-of-Two-Choices (PO2)—were designed for web services with relatively uniform request characteristics. LLM serving violates these assumptions: variable prompt lengths create high service-time variance, unpredictable generation lengths make completion times unknown at routing time, and continuous batching [2] introduces temporal dependencies between co-located requests. Most critically, *server heterogeneity* is becoming the norm rather than the exception. In production deployments, capacity asymmetries arise from mixed GPU generations (e.g., MI300X alongside MI250X), thermal throttling under sustained load, co-location interference, spot-instance availability, and deliberate over-provisioning strategies. This heterogeneity trend will only accelerate as organizations adopt disaggregated serving across diverse hardware pools [4, 19].

These challenges stem from a fundamental limitation: existing algorithms are *reactive*, responding to current state without anticipating future load. This motivates our key insight: **request routing for LLM serving is naturally a control problem** that benefits from model-based predictive reasoning. Model Predictive Control (MPC) [9] provides this capability: it uses a system model to predict future states over a finite horizon, optimizes control actions to minimize a cost function, applies the first action, then re-optimizes—the receding horizon principle.

We present **Orbit**, a framework that augments standard routing algorithms with MPC-based adaptive weighting. Rather than replacing proven heuristics like PO2, Orbit *wraps* them with a predictive layer that adjusts routing probabilities based on predicted queue trajectories. We validate Orbit on a production-class cluster of AMD Instinct MI300X GPUs with NixlConnector-based RDMA KV-cache transfer [7], moving beyond simulation-only evaluation to real-system validation. Our experiments reveal a nuanced picture: MPC is not universally beneficial, but provides substantial gains under specific conditions.

Contributions.

1. We formulate LLM request routing as an MPC problem over queue dynamics and design an efficient QP solver suitable for real-time operation (§3.2–3.4).
2. We validate Orbit on a real MI300X cluster with RDMA-based KV-cache transfer, conducting 40 experiments across five evaluation phases covering

concurrency scaling, algorithm comparison, heterogeneous configurations, scale-out topologies, and variable workloads (§4–5).

3. We provide an honest characterization of when MPC helps and when it does not: 45.8% tail latency reduction at heterogeneous 3P3D scale, 49.8% for prefill-heavy workloads, but 2–5% overhead in homogeneous settings (§5).
4. We identify PO2 as a surprisingly strong baseline for disaggregated serving, achieving 66% tail latency reduction over RR without any predictive machinery (§5.2).

2 Background and Motivation

2.1 Prefill-Decode Disaggregation

Modern LLM inference proceeds in two phases. The *prefill* phase processes all input tokens in parallel, computing attention over the full prompt and populating the KV-cache. This phase is compute-intensive, with runtime proportional to prompt length. The *decode* phase generates output tokens autoregressively, each step attending to all previous tokens via cached key-value pairs. Decode is memory-bandwidth limited.

Systems like vLLM [1] introduced PagedAttention for efficient KV-cache memory management, enabling high throughput through continuous batching [2]. Splitwise [4] and DistServe [3] extended this with prefill-decode disaggregation, assigning phases to specialized hardware. After prefill completes, the KV-cache must be transferred to a decode server—a step that introduces non-trivial overhead unless high-bandwidth interconnects are available. NIXL [7] and its NixlConnector integration with vLLM enable zero-copy KV-cache transfer over RDMA (InfiniBand/RoCE), reducing transfer latency to sub-millisecond levels for typical cache sizes.

2.2 Routing Policies and Their Limitations

The vLLM router [6] supports several load balancing strategies:

- **Round Robin (RR):** Distributes requests cyclically. Simple but ignores server load and capacity differences.
- **Random:** Uniform random selection. No load awareness.
- **Power-of-Two-Choices (PO2):** Samples two random servers and routes to the less loaded one. Achieves exponentially better tail latency than random under idealized conditions [8].

PO2’s theoretical optimality assumes [8]: (i) homogeneous servers, (ii) Poisson arrivals, (iii) exponential service times, and (iv) instantaneous queue depth information. LLM serving violates all four. Yet as our experiments show (§5.2), PO2 remains remarkably effective in practice—its queue-awareness provides substantial tail-latency benefits even under violated assumptions. The question is whether additional predictive capability can improve upon PO2 when server capacities differ significantly.

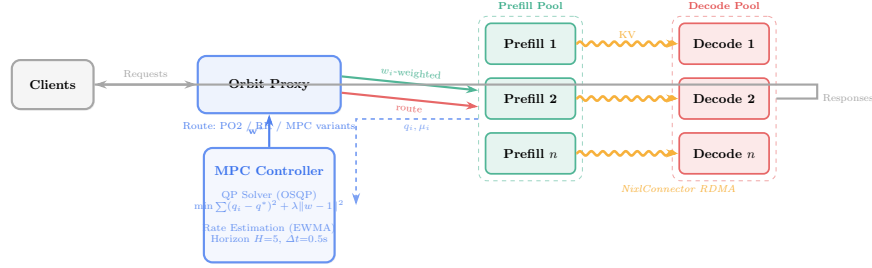


Fig. 1: **Orbit system architecture.** The MPC Controller runs as a sidecar, collecting queue depths and service rates from servers via periodic polling. It solves a QP to compute optimal routing weights $\mathbf{w}^*$ which modulate how the proxy distributes requests across the prefill pool. After prefill, KV-cache state is transferred via NixlConnector over RDMA to the decode pool for token generation.

2.3 The Heterogeneity Challenge

Consider a disaggregated deployment where some servers have reduced capacity—due to GPU generation differences, thermal throttling, memory pressure from co-located workloads, or intentional over-provisioning. Round Robin sends equal traffic to unequal servers, causing the slower ones to accumulate queues. PO2 mitigates this by routing to less-loaded servers, but its reactive nature means it detects overload only after queues have already grown.

The fundamental issue is *information asymmetry*: queue depth is a lagging indicator of server load. A server with 10 pending requests that are 90% complete is less loaded than one with 5 requests that just arrived. This motivates a predictive approach that models how queues *will* evolve rather than just observing where they *are*.

3 Orbit Framework

3.1 System Architecture

Figure 1 presents the Orbit architecture within a disaggregated serving deployment. Orbit operates as a sidecar process alongside the request proxy, computing MPC weight adjustments at regular intervals and feeding them to the routing layer.

3.2 System Model

We model each server $i \in \{1, \dots, n\}$ as a queue with discrete-time dynamics:

$$q_i(k+1) = \max(0, q_i(k) + \Delta t (\hat{a} \cdot p_i(k) - \mu_i(k))) \quad (1)$$

where $q_i(k)$ is the predicted queue length at step k , $\hat{a}$ is the estimated global arrival rate, $\mu_i(k)$ is the estimated service rate of server i , $p_i(k) = w_i(k) / \sum_j w_j(k)$ is the routing fraction determined by normalized weights $w_i(k) \in [w_{\min}, w_{\max}]$, and Δt is the controller timestep.

Rate Estimation. Service rates are estimated via exponential moving average:

$$\mu_i(k) = \alpha \cdot \frac{1}{\tau_i^{\text{recent}}} + (1 - \alpha) \cdot \mu_i(k - 1) \quad (2)$$

where τ_i^{recent} is the average service time of recent completions, and $\alpha = 0.3$ controls adaptation speed. Arrival rates are similarly estimated from observed request counts over a sliding window.

3.3 MPC Formulation

At each control timestep, Orbit solves:

$$\min_{\{w_i^{(k)}\}} \sum_{k=0}^{H-1} \sum_{i=1}^n \left[\left(q_i^{(k+1)} - q^* \right)^2 + \lambda \left(w_i^{(k)} - 1 \right)^2 \right] \quad (3)$$

$$\text{s.t.} \quad q_i^{(k+1)} = q_i^{(k)} + \Delta t (\hat{a} \cdot w_i^{(k)} / n - \mu_i) \quad (4)$$

$$w_{\min} \leq w_i^{(k)} \leq w_{\max} \quad \forall i, k \quad (5)$$

The queue tracking term drives predicted queue lengths toward target $q^* = H$, while the regularization term (coefficient $\lambda = 0.5$) keeps weights near the baseline value of 1.0, preventing aggressive oscillation when the system model is inaccurate. Weight bounds $[w_{\min}, w_{\max}] = [0.1, 3.0]$ prevent starvation while allowing up to $3\times$ redistribution.

Note that Eq. (4) uses the simplified routing fraction w_i/n (with constraint $\sum_i w_i = n$) rather than $w_i / \sum_j w_j$ to maintain Disciplined Convex Programming (DCP) compliance, enabling the use of efficient QP solvers.

3.4 Efficient Solver

After substituting the dynamics constraint into the objective, the problem reduces to a convex QP. We solve it using OSQP [15] via the `cvxpy` interface with warm-starting from the previous timestep's shifted solution. For $n \leq 8$ servers and horizon $H = 5$, solve time is under 1ms. Fallback behavior sets all weights to 1.0 if the solver fails or times out, reverting to baseline algorithm behavior.

3.5 Integration with Routing Algorithms

Orbit operates as a modular wrapper around existing routers. For PO2, the modification is minimal: instead of comparing raw queue depths, we compare *weighted* depths. Algorithm 2 shows the integration.

Algorithm 1 Orbit MPC Controller

-
- 1: **Parameters:** Horizon H , timestep Δt , target q^* , regularization λ
 - 2: **State:** Weights $\mathbf{w} = [w_1, \dots, w_n]$, rates $\boldsymbol{\mu}$
 - 3: Initialize $w_i \leftarrow 1.0$ for all servers i
 - 4: **loop**
 - 5: Observe queue depths $\mathbf{q}$ and update rate estimates $\boldsymbol{\mu}, \hat{a}$
 - 6: Solve QP (3)–(5) $\rightarrow \mathbf{w}^*$
 - 7: Apply first-step weights: $w_i \leftarrow w_i^{*(0)}$
 - 8: **yield** $\mathbf{w}$ to routing layer
 - 9: Sleep for Δt
 - 10: **end loop**
-

Algorithm 2 MPC-Augmented Power-of-Two-Choices

-
- 1: **Input:** Request r , server pool $\mathcal{S}$, MPC weights $\mathbf{w}$
 - 2: Sample two servers $s_1, s_2 \sim \text{Uniform}(\mathcal{S})$
 - 3: Get queue depths q_1, q_2
 - 4: Compute scores: $\text{score}_i = q_i/w_i$ for $i \in \{1, 2\}$
 - 5: Route r to $\arg \min_i \text{score}_i$
-

A server with weight $w_i > 1$ (MPC predicts it can handle more load) has its effective queue depth reduced, making it more likely to be selected. For Round Robin integration, we replace uniform cycling with sampling according to the normalized weight distribution $p_i = w_i / \sum_j w_j$.

4 Cluster Experiment Setup

4.1 Hardware and Software

We evaluate Orbit on a production-class cluster of AMD Instinct MI300X accelerators. Each node houses 8 MI300X GPUs connected via high-bandwidth Infinity Fabric, with inter-node communication over InfiniBand using Mellanox ConnectX adapters. Table 1 summarizes the configuration.

The serving stack uses vLLM 0.14.0 with disaggregated prefill-decode execution. Each request follows a two-phase protocol: the proxy forwards the request to a prefill server with the `do_remote_decode` flag; after prefill, the server transfers KV-cache state via NixlConnector to a decode server using zero-copy RDMA, and the decode server generates tokens using the transferred cache. An `etcd` cluster coordinates server discovery and KV transfer metadata.

4.2 Heterogeneous Configuration Methodology

A key aspect of our evaluation is testing under controlled heterogeneity. Rather than requiring different GPU hardware, we simulate capacity asymmetries using vLLM’s runtime parameters:

Table 1: Cluster configuration.

Component	Specification
GPU	AMD Instinct MI300X (192 GB HBM3)
Nodes	8 nodes, 8 GPUs each
Interconnect	InfiniBand (mlx5, RDMA)
Tensor Parallelism	TP=8 (full node per server instance)
vLLM Version	0.14.0 with NixlConnector
KV Transfer	NixlConnector over UCX/RDMA
Model	Qwen/Qwen2.5-14B-Instruct
Scheduler	Slurm (node-level allocation)

- **Normal nodes:** `--gpu-memory-utilization 0.70, --max-num-seqs 256`
- **Throttled nodes:** `--gpu-memory-utilization 0.45, --max-num-seqs 64`

Reducing `gpu-memory-utilization` limits the available KV-cache memory, constraining batch capacity. Reducing `max-num-seqs` directly caps concurrent request processing. Together, these create approximately $2\text{--}3\times$ effective capacity difference between normal and throttled nodes, which is representative of real-world heterogeneity from mixed GPU generations, thermal throttling, or co-location interference [4, 19].

Figure 2 illustrates the two topologies we evaluate: 2-prefill/2-decode (2P2D) and 3-prefill/3-decode (3P3D), with alternating normal and throttled nodes.

4.3 Experiment Design

We organize evaluation into five phases, each isolating a specific dimension:

1. **Phase 1 – Concurrency Sweep:** 2P2D homogeneous, varying concurrency from 2 to 32. Establishes baseline MPC overhead.
2. **Phase 2 – Algorithm Comparison:** All six routing policies (RR, Random, PO2, MPC+RR, MPC+PO2, vLLM built-in RR) at concurrency 8 and 16. Characterizes policy landscape.
3. **Phase 3 – Heterogeneous 2P2D:** Normal + throttled server pairs. Tests MPC value under capacity asymmetry.
4. **Phase 4 – Heterogeneous 3P3D:** Three prefill + three decode servers (7 nodes total), with two-thirds throttled. Tests scaling behavior.
5. **Phase 5 – Variable ISL/OSL:** Four workload shapes (prefill-heavy to decode-heavy). Tests adaptability.

Metrics. We report throughput (tokens/s), mean and P99 time-per-output-token (TPOT), and P99 inter-token latency (ITL). All experiments use ISL=256, OSL=256 unless otherwise noted.

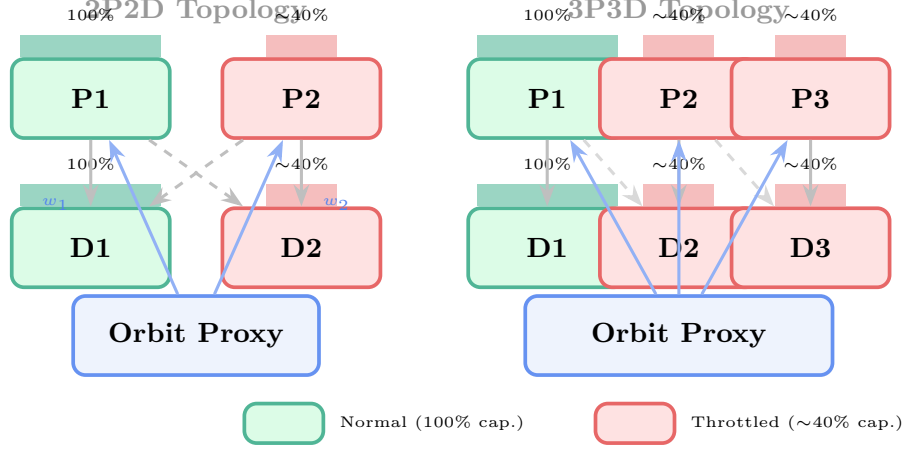


Fig. 2: **Heterogeneous topologies.** Capacity bars indicate relative throughput capacity. In 2P2D, one of each pair is throttled. In 3P3D, two of three are throttled, creating greater imbalance. Solid arrows: primary KV transfer paths; dashed: cross-pair transfers.

Table 2: Phase 1: Concurrency sweep (2P2D homogeneous, ISL=OSL=256). MPC adds overhead without benefit when servers are identical.

Con.	Throughput (tok/s)		TPOT_P99 (ms)		ITL_P99 (ms)	
	RR	MPC	RR	MPC	RR	MPC
2	362	354	3.98	4.25	7.67	11.24
4	555	527	4.42	4.54	8.82	12.78
8	757	729	4.89	5.38	12.82	16.57
16	1369	1327	5.81	6.49	32.37	29.83
32	2498	2371	6.99	8.18	34.14	31.41

5 Experimental Results

5.1 Phase 1: MPC Overhead in Homogeneous Settings

Table 2 compares RR against MPC-augmented RR across concurrency levels in a homogeneous 2P2D configuration. MPC consistently adds 2–5% throughput overhead, with no latency benefit. At concurrency 32, the gap is most pronounced: MPC_RR achieves 2371 tok/s versus 2498 tok/s for plain RR (−5.1%). The overhead stems from periodic QP solves (every 0.5s) and routing non-uniformity during weight convergence.

This is a deliberately negative result. It establishes that MPC is not a universal improvement and should not be enabled blindly. At high concurrency (≥ 16), MPC shows marginal ITL improvement (29.83 vs 32.37 at Con=16), hinting

Table 3: Phase 2: Routing algorithm comparison (2P2D homogeneous, 128 prompts). PO2 achieves dramatically better tail latency at Con=16.

Policy	Con=8		Con=16	
	Tput (tok/s)	ITL_P99 (ms)	Tput (tok/s)	ITL_P99 (ms)
RR (built-in)	1039	12.19	1392	31.34
RR (Orbit)	1029	13.21	1391	32.88
Random	981	29.94	1366	32.48
PO2	1036	12.33	1380	10.61
MPC+RR	983	16.89	1345	28.96
MPC+PO2	1009	15.94	1360	17.09

that predictive routing helps under contention—a theme explored further in later phases.

5.2 Phase 2: Routing Algorithm Landscape

Table 3 compares all six routing policies at concurrency 8 and 16 in a homogeneous 2P2D configuration.

The standout finding is PO2’s performance at Con=16: ITL_P99 of 10.61ms versus 31.34ms for RR—a **66% reduction**. This is a much larger improvement than PO2’s theoretical analysis would predict for only two servers, and suggests that even with two choices, load-aware routing provides substantial benefit when batching creates queue-depth variation.

MPC+PO2 achieves 17.09ms ITL_P99 at Con=16—better than RR (31.34ms) but worse than pure PO2 (10.61ms). The MPC overhead dilutes PO2’s natural load-awareness in this homogeneous setting. Random routing performs worst, confirming that load-blindness is costly under contention.

5.3 Phase 3: Heterogeneous 2P2D

Table 4 presents results with one normal and one throttled server in each pool. PO2 remains the strongest policy, achieving 10.27ms ITL_P99 at Con=8 (19.6% better than RR’s 12.77ms) and 10.88ms at Con=16 (65% better than RR’s 31.04ms).

With only two servers per pool, PO2 always samples both servers and always picks the less loaded one—effectively becoming Join-Shortest-Queue, which is optimal for this case. MPC’s predictive overhead cannot improve upon this direct load observation. The natural question is whether this changes at larger scale, where PO2 samples only a fraction of servers.

5.4 Phase 4: Heterogeneous 3P3D Scale-out

With three prefill and three decode servers (two-thirds throttled), the dynamics change. Table 5 shows the results.

Table 4: Phase 3: Heterogeneous 2P2D. PO2 dominates; MPC+PO2 improves over RR but not over pure PO2.

Policy	Con.	Tput (tok/s)	TPOT_P99 (ms)	ITL_P99 (ms)
RR	8	996	5.73	12.77
PO2	8	1016	4.69	10.27
MPC+PO2	8	1013	5.21	15.23
RR	16	1386	5.62	31.04
PO2	16	1388	5.62	10.88
MPC+PO2	16	1342	6.15	17.82

Table 5: Phase 4: Heterogeneous 3P3D. At Con=16, MPC+PO2 achieves the best results across all metrics.

Policy	Con.	Tput (tok/s)	TPOT_mean (ms)	TPOT_P99 (ms)	ITL_P99 (ms)
RR	8	1036	4.47	4.78	10.55
MPC+PO2	8	1059	4.41	4.85	15.71
RR	16	1400	5.19	6.04	31.29
MPC+PO2	16	1412	4.97	5.60	16.95

At Con=16, MPC+PO2 improves on RR across every metric:

- Throughput: +0.9% (1412 vs 1400 tok/s)
- TPOT_mean: −4.2% (4.97 vs 5.19ms)
- TPOT_P99: −7.3% (5.60 vs 6.04ms)
- ITL_P99: **−45.8%** (16.95 vs 31.29ms)

This is where MPC’s value becomes clear. With three servers, PO2 only samples two of three—there is a 1/3 probability of missing the least-loaded server entirely. MPC compensates by proactively steering traffic based on predicted queue evolution, avoiding the sampling limitation of PO2. The 45.8% ITL_P99 reduction is the headline result: at scale with heterogeneous servers and moderate-to-high load, predictive routing provides substantial tail-latency improvement.

At Con=8, MPC+PO2 achieves 2.2% higher throughput and 1.3% better TPOT_mean, but its ITL_P99 is higher (15.71 vs 10.55ms). At lower load, there is less contention to predict, and MPC’s periodic weight adjustments can temporarily create imbalance.

Figure 3 illustrates a key pattern: RR’s ITL_P99 degrades sharply at Con=16 (jumping to ~31ms), while MPC+PO2 maintains relatively stable tail latency (~16–17ms) across configurations. This stability under varying conditions is a practical advantage for meeting SLA targets.

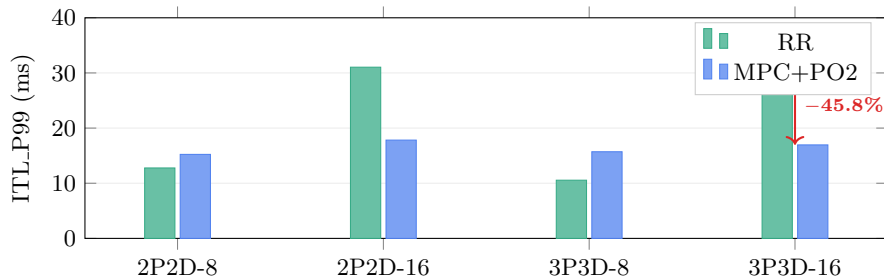


Fig. 3: **ITL_P99 across heterogeneous configurations.** MPC+PO2 maintains stable tail latency (~ 16 ms) regardless of scale and concurrency, while RR degrades sharply at Con=16.

Table 6: Phase 5: Variable ISL/OSL (2P2D homogeneous, Con=8). MPC excels for prefill-heavy workloads.

Workload	Throughput (tok/s)		ITL_P99 (ms)		
	RR	MPC	RR	MPC	Δ ITL
ISL=64, OSL=512	1354	1317	9.33	13.41	+43.7%
ISL=128, OSL=256	1062	996	15.51	16.01	+3.2%
ISL=256, OSL=128	720	717	30.58	15.34	-49.8%
ISL=512, OSL=64	414	413	46.36	36.28	-21.7%

5.5 Phase 5: Variable Workload Shapes

Table 6 evaluates MPC under four workload profiles with different input/output length ratios.

The pattern is striking: MPC provides significant ITL_P99 reduction for prefill-heavy workloads (ISL > OSL) but hurts for decode-heavy ones (ISL < OSL). At ISL=256/OSL=128, MPC achieves **49.8% lower ITL_P99** (15.34 vs 30.58ms). At ISL=512/OSL=64, the reduction is 21.7%.

The explanation is that prefill time scales with input length and exhibits high variance across requests. With long prompts, some prefill servers temporarily become much slower (processing large prefills) while others drain their queues. MPC’s queue prediction detects this imbalance forming and proactively reroutes traffic before queues diverge. For decode-heavy workloads, token generation times are more uniform (each step involves similar-sized attention), so there is less variance for MPC to exploit, and its overhead dominates.

5.6 Simulation Validation

To complement the cluster experiments, we run the same configurations through our discrete-event simulator (Table 7), which models disaggregated serving with configurable heterogeneous delays. The simulator uses the same MPC formulation but with synthetic service-time distributions.

Table 7: Simulation results (2P2D, $4\times$ heterogeneity). Consistent with cluster trends: MPC improves mean and tail latency under heterogeneity.

Method	Mean (ms)	P99 (ms)	Std (ms)	Impr.
Baseline PO2	659	5283	936	—
MPC-PO2	552	3526	624	+16.2%
<i>Bursty workload ($3\times$ periodic spikes)</i>				
Baseline PO2	639	5438	972	—
MPC-PO2	632	3407	665	+37.3% (P99)

Table 8: Deployment recommendations based on experimental findings.

Scenario	Recommended Rationale	
Homogeneous, any load	PO2	Best tail latency, zero overhead
Hetero, 2 servers/pool	PO2	PO2 = JSQ with 2 servers
Hetero, ≥ 3 servers	MPC+PO2	Compensates PO2 sampling
Prefill-heavy workloads	MPC+PO2	Exploits prefill variance
Decode-heavy workloads	RR or PO2	Uniform decode $\Rightarrow$ no MPC value

The simulation results are directionally consistent with cluster experiments: MPC provides 16.2% mean latency improvement and 33% P99 reduction under $4\times$ heterogeneity. Under bursty conditions, the P99 improvement reaches 37.3%. The larger relative improvements in simulation reflect the higher heterogeneity ratio ($4\times$ vs $\sim 2\text{--}3\times$ in the cluster) and the cleaner service-time model. The qualitative agreement between simulation and cluster results increases confidence that the MPC framework captures real load-balancing dynamics, not just simulation artifacts.

6 Discussion

When MPC Works—and When It Does Not. Our five-phase evaluation paints a nuanced picture. Table 8 summarizes the deployment guidance.

The key insight is that MPC’s value grows along two axes: (1) the degree of heterogeneity or workload variance, and (2) the scale of the server pool. With 2 servers per pool, PO2 samples both and is effectively optimal. With 3+ servers, PO2’s random sampling misses capacity information, and MPC fills this gap. This suggests that MPC’s benefit will grow further with larger pools (e.g., 8P8D, 16P16D), which we leave for future work.

The PO2 Surprise. One of the more interesting findings is PO2’s effectiveness. At Con=16 in a homogeneous 2P2D setup, PO2 reduces ITL_P99 by 66% over RR (10.61 vs 31.34ms). This is substantially better than the theoretical prediction for two-server systems, likely because vLLM’s continuous batching creates bursty queue-depth variation that PO2 exploits well. This finding has immediate

practical value: for deployments not yet ready to adopt MPC, simply switching from RR to PO2 yields large tail-latency improvements at zero cost.

Heterogeneity as the Future. We deliberately focus on heterogeneous configurations because we believe they represent the future of large-scale LLM serving. Several trends drive this:

- **Mixed GPU generations:** Organizations accumulate hardware over time; serving fleets will include MI300X alongside MI250X or future MI400 accelerators.
- **Spot/preemptible instances:** Cost optimization drives use of spot instances with variable availability and potentially different hardware.
- **Thermal throttling:** Sustained inference loads cause thermal throttling that reduces effective capacity non-uniformly across a cluster.
- **Co-location interference:** Sharing nodes with training or other inference workloads creates unpredictable capacity reduction.

Our controlled heterogeneity methodology—throttling via `gpu-memory-utilization` and `max-num-seqs`—provides a reproducible way to study these effects without requiring diverse hardware. The $2\text{--}3\times$ capacity difference we simulate is conservative compared to the $4\text{--}10\times$ differences possible between GPU generations [4].

Limitations.

1. **Single model:** We evaluate only Qwen2.5-14B. Different model sizes may exhibit different prefill/decode time ratios, affecting when MPC helps. Larger models with longer prefill times may show greater MPC benefit.
2. **Synthetic workload:** Our benchmarks use fixed ISL/OSL pairs. Production workloads follow skewed distributions with temporal correlation [4]. ShareGPT or production trace evaluation would strengthen the results.
3. **Limited scale:** We evaluate up to 3P3D (7 nodes). Production deployments may use $10\text{--}100\times$ more servers, where MPC’s advantage over PO2 should grow but where solve time also increases.
4. **No cache-aware integration:** We do not explore combining MPC with cache-aware routing, which could provide complementary benefits for multi-turn conversations.
5. **Controller overhead:** While the QP solver runs in under 1ms, the Python-based proxy introduces latency compared to vLLM’s Rust router. A production implementation should integrate MPC into the compiled router.

7 Related Work

LLM Serving Systems. vLLM [1] introduced PagedAttention for efficient KV-cache management. Orca [2] proposed continuous batching for higher throughput. Splitwise [4] and DistServe [3] developed prefill-decode disaggregation. SarathiServe [5] addresses stall-free scheduling with chunked prefills. DynamoLLM [19] studies energy-efficient serving with heterogeneous GPU configurations. The vLLM router [6] provides production load balancing with PO2, RR, and cache-aware policies.

KV-Cache Transfer. NIXL [7] enables zero-copy KV-cache transfer over RDMA interconnects, a critical enabler for disaggregated serving. Mooncake [23] proposes a KV-cache-centric architecture for disaggregated serving. These systems make the transfer overhead small enough that routing quality becomes the dominant factor in end-to-end latency.

Load Balancing Theory. The Power-of-Two-Choices [8] achieves exponential improvement in maximum load under idealized conditions. Join-Shortest-Queue [17] is optimal for homogeneous servers but requires global state. Consistent hashing [16] provides stateless routing with cache affinity. Recent work on learned load balancing [18] uses reinforcement learning but requires extensive training data and may not generalize across workloads.

Model Predictive Control. MPC [9, 10] has been applied to network congestion control [12], data center thermal management [13], and cloud resource allocation [14]. Williams et al. [11] developed sampling-based MPC for robotics. To our knowledge, Orbit is the first MPC application to LLM serving request routing. Unlike RL-based approaches [18], MPC requires no pre-training and provides interpretable optimization objectives.

Heterogeneous Cluster Scheduling. Gavel [20] addresses heterogeneity-aware scheduling for DNN training. Clockwork [21] provides predictable serving through careful scheduling. Cortez et al. [22] study workload prediction for cloud resource management. Our work complements these by addressing the specific routing challenges in disaggregated inference with asymmetric server capacities.

8 Conclusion

We presented Orbit, a Model Predictive Control framework for adaptive request routing in disaggregated LLM serving. Through 40 experiments on an AMD MI300X cluster with RDMA-based KV-cache transfer, we provide a nuanced evaluation: MPC adds 2–5% overhead in homogeneous settings where PO2 already excels, but delivers 45.8% tail-latency reduction at heterogeneous 3P3D scale and 49.8% reduction for prefill-heavy workloads. We also identify PO2 as a surprisingly strong baseline, achieving 66% ITL_P99 reduction over Round Robin with zero overhead—an immediately actionable finding for production deployments.

The key insight is that MPC’s value scales with heterogeneity and pool size: with 2 servers, PO2’s exhaustive sampling is sufficient; with 3+ servers and capacity asymmetry, MPC’s predictive capability compensates for PO2’s sampling limitation. As disaggregated LLM serving grows in scale and hardware diversity, we expect the conditions favoring MPC to become increasingly common.

Future work includes evaluation with additional models and production traces, scaling to larger server pools where MPC’s advantage should grow, integration with cache-aware routing, and implementation within vLLM’s compiled Rust router for production-grade latency.

Acknowledgments. We thank the anonymous reviewers for their feedback. Experiments were conducted on AMD internal cluster infrastructure.

References

1. Kwon, W., et al.: Efficient memory management for large language model serving with PagedAttention. In: SOSP (2023)
2. Yu, G., et al.: Orca: A distributed serving system for transformer-based generative models. In: OSDI (2022)
3. Zhong, Y., et al.: DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: OSDI (2024)
4. Patel, P., et al.: Splitwise: Efficient generative LLM inference using phase splitting. In: ISCA (2024)
5. Agrawal, A., et al.: Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In: OSDI (2024)
6. vLLM Project: vLLM Router: A high-performance load balancer for large-scale serving. <https://github.com/vllm-project/router> (2025)
7. NVIDIA: NIXL: Network Interface eXchange Library for zero-copy KV-cache transfer. <https://github.com/ai-dynamo/nixl> (2024)
8. Mitzenmacher, M.: The power of two choices in randomized load balancing. IEEE TPDS 12(10), 1094–1104 (2001)
9. Camacho, E.F., Bordons, C.: Model Predictive Control. Springer (2013)
10. Garcia, C.E., Prett, D.M., Morari, M.: Model predictive control: Theory and practice. Automatica 25(3), 335–348 (1989)
11. Williams, G., et al.: Model predictive path integral control using covariance variable importance sampling. arXiv:1707.02342 (2017)
12. Low, S.H., Lapsley, D.E.: Optimization flow control—I: Basic algorithm and convergence. IEEE/ACM ToN 7(6), 861–874 (1999)
13. Lazic, N., et al.: Data center cooling using model-predictive control. In: NeurIPS (2018)
14. Zhang, Y., et al.: Model predictive control for cloud resource management. In: ICDCS (2021)
15. Stellato, B., et al.: OSQP: An operator splitting solver for quadratic programs. Mathematical Programming Computation 12(4), 637–672 (2020)
16. Karger, D., et al.: Consistent hashing and random trees. In: STOC (1997)
17. Eschenfeldt, P., Gamarnik, D.: Join the shortest queue with many servers. Mathematics of OR 43(3), 867–886 (2018)
18. Mao, H., et al.: Learning scheduling algorithms for data processing clusters. In: SIGCOMM (2019)
19. Stojkovic, J., et al.: DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. arXiv:2408.00741 (2024)
20. Narayanan, D., et al.: Heterogeneity-aware cluster scheduling policies for deep learning workloads. In: OSDI (2020)
21. Gujarati, A., et al.: Serving DNNs like clockwork: Performance predictability from the bottom up. In: OSDI (2020)
22. Cortez, E., et al.: Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In: SOSP (2017)
23. Qin, R., et al.: Mooncake: A KVCache-centric disaggregated architecture for LLM serving. arXiv:2407.00079 (2024)
24. Dang, Y., et al.: AIOPS: Real-world challenges and research innovations. In: ICSE-SEIP (2019)